

Let's consider the following scenario. Say you have a certain amount of work that takes an hour to complete. If the workload doubles, how long would it take then? You might be tempted to think it takes twice as long, but actually, the answer depends on the kind of work that's being done.

If it takes an hour to read a short book, it will take more or less two hours to read two short books. But if you can alphabetize 500 books in an hour, alphabetizing 1,000 books will most likely take longer than two hours, because you have to find the correct place for each book in a much larger collection of books. On the other hand, if you're just checking whether or not a bookshelf is empty, it doesn't matter if there are 0, 10, or 1,000 books on the shelf. One glance and you'll immediately know the answer. The runtime remains roughly constant no matter how many books there are. Although some people might be faster or slower at reading or alphabetizing books, these general trends remain the same.

The big O of the algorithm describes these trends. An algorithm can run on a fast or slow computer, but we can still use big O to describe how well an algorithm performs in general, regardless of the actual hardware executing the algorithm. Big O doesn't use specific units, such as seconds or CPU cycles, to describe an algorithm's runtime, because these would vary between different computers or programming languages.

Big O Orders

Big O notation commonly defines the following orders. These range from the *lower* orders, which describe code that, as the amount of data grows, slows down the least, to the *higher* orders, which describe code that slows down the most:

1. $O(1)$, Constant Time (the lowest order)
2. $O(\log n)$, Logarithmic Time
3. $O(n)$, Linear Time
4. $O(n \log n)$, N-Log-N Time
5. $O(n^2)$, Polynomial Time